

Nombre y apellido: LU: DNI:
E-mail: # Hojas: Firma:

Poner nombre, fecha y firma en cada hoja. Justificar todas las respuestas.

1.a	1.b	1.c	1.d	2.a	2.b	2.c	2.d	3.a	3.b	3.c	3.d	Nota

Ejercicio 1 (Cálculo- λ). Sean M, N dos programas (es decir, términos cerrados y tipables) que además tienen el mismo tipo. Decimos que M es *distinguable* de N si existe una lista finita de valores $V_1, \dots, V_n$, con $n \geq 0$, tales que $M V_1 \dots V_n \rightarrow^* V_a$ y $N V_1 \dots V_n \rightarrow^* V_b$, con $V_a \neq V_b$.

- Demostrar que el programa $\lambda f : \text{Bool} \rightarrow \text{Bool}. f (f \text{ true})$ es distinguishable del programa $\lambda f : \text{Bool} \rightarrow \text{Bool}. f \text{ true}$.
- Decimos que M es indistinguishable de N si no vale que M es distinguishable de N . Es fácil de ver que M es indistinguishable de M . Exhibir dos programas M y N *distintos*, tales que M sea indistinguishable de N .
- Decidir si la siguiente afirmación es verdadera o falsa y justificar: si M es distinguishable de N , entonces N es distinguishable de M .
- Decidir si la siguiente afirmación es verdadera o falsa y justificar:
si M es distinguishable de N , con $M, N : T$, entonces $\lambda f : T \rightarrow \text{Bool}. f M$ es distinguishable de $\lambda f : T \rightarrow \text{Bool}. f N$.

Ejercicio 2 (Programación Lógica y Resolución).

Se cuenta con la siguiente base de conocimientos sobre relaciones familiares:

```
progenitor(diego,dalma).      progenitor(chitoto,ana).      pareja(chitoto,tota).
progenitor(diego,gianinna).   progenitor(ana,daniel).      pareja(diego,claudia).
progenitor(tota,diego).       pareja(gianinna,osvaldo).     pareja(ana,pedro).
```

```
pareja(X,Y) :- pareja(Y,X).
ancestro(A, X) :- progenitor(A, X).
ancestro(A, X) :- progenitor(A, Y), ancestro(Y, X).
```

Donde $\text{progenitor}(P, H)$ indica que P es progenitor de H , $\text{pareja}(X, Y)$ indica que X e Y están en pareja, y $\text{ancestro}(A, X)$ indica que A es un ancestro (directo o indirecto) de X .

- Definir un predicado $\text{descendientes}(+P, -L)$, que instancie en L la lista de todos los descendientes de P , sin repetidos. Obs.: No se exige ningún orden particular en la lista resultante.
- Analizar y justificar detalladamente si uno o ambos argumentos del predicado anterior pueden ser reversibles.
- Definir un predicado $\text{ancestroComunMasCercano}(+P1, +P2, -A)$, que se satisfaga cuando A es el ancestro común más cercano entre $P1$ y $P2$. Si hay más de uno, debería dar todos ellos.
Por ejemplo, si dos personas son hermanas, sus ancestros comunes más cercanos serían sus progenitores.
- Comparar qué pasa en Prolog y en Resolución General ante una fórmula como la de abajo, y explicar en detalle por qué podrían haber diferencias:

$\exists A. (\text{ancestro}(A, \text{daniel}) \wedge \text{ancestro}(A, \text{gianinna}))$

(El examen sigue al dorso $\rightarrow$)

Ejercicio 3 (Programación funcional). El tipo algebraico `AdjAB a` describe a las *adjunciones de árboles binarios* con elementos de tipo `a`. Las `AdjAB` representan listas de árboles binarios, donde cada uno de ellos está al mismo nivel que el otro, que pueden o no tener una raíz en común de acuerdo al constructor usado.

```
data AdjAB a = Raiz a [AB a] | Adj [AB a]
data AB a = Nil | Hoja a | Arb a (AB a) (AB a)
```

Por ejemplo, las siguientes expresiones denotan distintos árboles de tipo `AdjAB Int`:

- `Adj [Nil, Nil, Arb 3 Nil Nil, Hoja 6]`
- `Raiz 5 [Arb 3 (Arb 1 Nil Nil) (Hoja 4), Arb 3 Nil Nil, Nil]`
- `Raiz 10 []`

- a) Dar el tipo y definir una función `foldAdjAB`, que abstraiga el esquema de recursión estructural sobre las *adjunciones de árboles binarios*, recorriendo todos los árboles por completo.
- b) Sin usar recursión explícita, dar el tipo y definir una función `mapAdjAB` que aplique una función dada a todos los valores de una adjunción.
- c) Dar el tipo y definir `recAdjAB` que abstraiga el esquema de recursión primitiva sobre las adjunciones, recorriendo todos los árboles por completo.
- d) Sin usar recursión explícita, dar el tipo y definir la función `ordenado`, que permita saber si toda la adjunción está ordenada. Así, cada árbol debe estar ordenado como un ABB y, adicionalmente, si hubiera raíz común, esta debe ser mayor que la raíz de todos los árboles. En el caso de `Nil`, este árbol está trivialmente ordenado y, además, se lo puede ignorar para comparar con la raíz común.